Mixed fast matrix multiplication: fused levels of accurate square and rectangular schemes over a vendor GEMM, chosen by a calibrated planner

Greg Sidebottom

Draft — October 28, 2026

Abstract

A fast matrix multiplication in practice is a few levels of a Strassen-like scheme over a vendor General Matrix Multiplication (GEMM) routine. We treat the levels as a menu: every level may use a different scheme, square or rectangular, and a plan is a sequence of levels ending in a GEMM leaf. On Apple's M4 Pro, with Accelerate's GEMM at the leaves, the menu is Strassen–Winograd $\langle 2\times 2\times 2 : 7\rangle$, the 262-operation accurate $\langle 4\times 4\times 4 : 48\rangle$ algorithm of Dumas, Pernet and Sedoglavic, and their accurate $\langle 3\times 3\times 6 : 40\rangle$ algorithm in its three orientations, which bring a square problem back to square after three levels. A level is three streaming passes whose cost is the number of blocks they move, so a scheme's worth is its saving of products per block moved and its amplification of the leaf error; a screen of 70 published schemes by these two measures leaves exactly this menu, and shows that nothing larger than 4×4 and no Kronecker composite can help. A calibrated dynamic program chooses the plan per shape, precision and error budget; it prices GEMM streams and pipelined passes that use the whole machine, and prices rectangular problems from the square calibration until they are calibrated themselves. At $n = 13824$ the planner's mixed plans run 1.73× (f32) and 1.91× (f64) faster than Accelerate's GEMM, at $n = 6912$ 1.38× and 1.71×. Measured against the exact product, the $3\times 3\times 6$-family plan that runs in the time of the best plan from the 4×4 scheme alone has 23–43% less error at every size and precision tried, and less than the GEMM's own. The planner's default budget, no less accurate than the GEMM, keeps every choice on that side: at 13824 f64 it finds a plan as fast as the Strassen-topped one, 1.9× over Accelerate, with half the GEMM's error. As a test of the opposite design – cheap passes instead of accurate ones – a ternary $\langle 3\times 3\times 6 : 42\rangle$ scheme from Perminov's catalogue, whose three programs need 139 additions against about 370 operations for the accurate scheme's, is measured as a level in the same library: its passes cost exactly the same (they are bound by the blocks they move), and its two extra products and 2.8× amplification make it slower and less accurate than the accurate level at every size tried. Apple's GPU, through the vendor's MPSGraph GEMM, ties the f32 plan at 13824 in time with 2.6× its error – its product is bit for bit Accelerate's – and is 1.4–1.6× faster below that size; it has no f64. Two implementations of the library, in C and in Rust, agree bit for bit.

1 Introduction

A scheme for multiplying a $g_1 \times g_2$ matrix by a $g_2 \times g_3$ matrix in $r < g_1 g_2 g_3$ products, applied to blocks, saves products at every level of recursion. On a real machine a level is not free: its linear combinations of blocks are passes over memory, and the products it hands down are General Matrix Multiplications (GEMMs, the BLAS routine) whose rate depends on their size and shape. Libraries that use fast schemes therefore apply one to three levels on top of a tuned GEMM and stop [8, 5].

Which scheme, how many levels, in what order, and at what cost in accuracy, are the questions this report answers for one machine, Apple's M4 Pro with Accelerate's GEMM, by measurement.

The approach is to mix. A *plan* is a sequence of *levels* from the top of the problem down to a GEMM leaf, and every level may come from a different scheme: Strassen–Winograd's $\langle 2\times 2\times 2 : 7\rangle$ (`SW`), the accurate $\langle 4\times 4\times 4 : 48\rangle$ algorithm of Dumas, Pernet and Sedoglavic [2, 3] in its 262-operation form [6, 20] (`D48`), and their accurate $\langle 3\times 3\times 6 : 40\rangle$ algorithm [4, 6] in its three orientations $3\times 3\times 6$, $3\times 6\times 3$ and $6\times 3\times 3$ (`R336`, `R363`, `R633`). The three orientations applied in turn divide a square problem by 54 and leave it square, where three 4×4 levels divide by 64. A plan such as `SW/R363+3/R633/G6` is one Strassen–Winograd level, then a $3\times 6\times 3$ level whose products are pipelined on three pass threads, then a $6\times 3\times 3$ level whose 40 products run as six single-threaded GEMM streams. Mixing pays for three reasons that the measurements separate:

1. *Order.* A level writes more data than it reads (the 4×4 scheme three times as much, Strassen's 1.75 times); the larger expansion belongs at the smaller level, so `SW/D48/G` moves $9.75\,n^2$ words per side where `D48/SW/G` moves $12.25\,n^2$ with the same 336 products.

2. *Shape.* A rectangular level hands down rectangular products; the $3\times 3\times 6$ cycle keeps 0.741 of the products per level against the 4×4 scheme's 0.75, moves $11.89\,n^2$ words against 12, stays under 58 streams per pass against 64, and after two levels its products are larger, so Accelerate runs them 13% faster.

3. *Accuracy.* Each level multiplies the leaf error by a factor set by the scheme and the point chosen in its orbit; the $3\times 3\times 6$ algorithm's factor is 0.74–0.79 of the 4×4 algorithm's per level in f32, and both are below one, because their products are shorter dot products than the GEMM's. Under an error budget an accurate level admits deeper, faster plans.

Two facts about the costs organize everything. First, a fused pass is bound by memory traffic, not arithmetic: the 201-operation program we found for the 4×4 scheme [19], DPS's original 315-operation program and their 284-operation accurate program all run at the same speed as fused passes (totals within 1%), so the operation counts that scheme searches minimize have little value here, while the error growth of the program matters [20]. Arithmetic shows only when a program is wasteful: our first programs for the $3\times 3\times 6$ scheme's R and P matrices, written as direct sums of the matrix rows, took 18 operations per output and their passes ran 30% slower than the traffic predicts; PLinOpt's optimizer, which finds the common subexpressions, cut those programs to a fifth of the operations and the passes to the predicted speed (Section 2). Second, the leaf GEMM dominates the time (85–95% of a plan), so a level pays only when its products are large enough for the GEMM to run near its peak, and a scheme that saves products beats one that saves additions. The ternary $\langle 3\times 3\times 6 : 42\rangle$ scheme of Section 7 is the controlled experiment: a third of the pass arithmetic, two products more, and it loses.

Contributions. (i) A cost model of fused levels in blocks moved and GEMMs priced inside the level that sits on them, calibrated per machine and precision, with predictions within 0.3–3% at $n \geq 8192$ (Section 2). (ii) A screen of 70 published schemes by saving per block moved and by leaf error amplification per 4×4-equivalent saving, which leaves the accurate $\langle 3\times 3\times 6 : 40\rangle$ cycle as the one addition to the 4×4 menu and rules out every larger shape and every Kronecker composite (Section 3). (iii) A per-level error model, validated within 25%, and the orbit floors of the amplification for the two accurate classes (Section 4). (iv) A planner over (m, k, n) shapes with GEMM streams, pipelines, padding to multiples of $2^a 3^b$, an error budget, and shape estimates that price uncalibrated rectangular problems within 2% (Section 5). (v) Measurements on the M4 Pro:

1.73× and 1.91× over Accelerate at $n = 13824$ in f32 and f64, with less error than the GEMM in both precisions for every plan the default budget admits, and 23–43% less than the best 4×4-only plan of the same speed; rectangular problems at 1.04–1.52×; and the GPU as the alternative, tied in time at 13824 f32 with 2.6× the error (Section 6). (vi) The ternary trial (Section 7). (vii) Two implementations, C and Rust, that agree in plans, predicted times and results bit for bit (Section 8). (viii) An assessment of the commercial value, in software and as a GPU or TPU feature (Section 10). Code, programs, calibrations and raw results are in the repository [21].

The result in two tables. Plans are written from the top of the problem down to the leaves, the levels separated by slashes. `D48` is a level of the accurate $\langle 4 \times 4 \times 4 : 48 \rangle$ scheme: it cuts A, B and C into 4×4 grids of blocks and forms the product with 48 block products instead of 64. `R336`, `R363` and `R633` are levels of the accurate $\langle 3 \times 3 \times 6 : 40 \rangle$ scheme in its three orientations, named by the grids they cut A and B into: `R363` cuts A 3×6 and B 6×3 and uses 40 block products instead of 54. `SW` is a Strassen–Winograd level (2×2 grids, 7 products instead of 8). `G` is the leaf, Accelerate's GEMM on the block products that remain, and `G6` runs those products as six concurrent single-threaded GEMM streams. `+p` after a level runs its passes – the sums that form the operands and accumulate the outputs – on p threads overlapped with the products below. So `R363+4/D48/G6` at $n = 13824$ is a $3 \times 6 \times 3$ level with four pass threads, whose 40 products of 4608×2304 by 2304×4608 each get a 4×4 level, whose $40 \times 48 = 1920$ leaf products of 1152×576 by 576×1152 go to Accelerate in six streams; the classical count would be $54 \times 64 = 3456$.

The error of a product is the largest entry of $C - C_{\text{exact}}$ divided by the largest entry of C_{exact}, where C_{exact} is the f64 product of the same inputs in f32 and a compensated double-double product in f64, on uniform random inputs in $[-1, 1)$; every method in a comparison sees the same inputs. The difference from Accelerate's own result is not an error measure, because Accelerate's error is as large as a plan's (Section 6).

Table 1 is the accuracy claim: in each precision and size, the $3 \times 3 \times 6$-family plan that runs in the time of the best plan built from the 4×4 scheme alone has 23–43% less error, and is 4–6% faster besides. Table 2 is the comparison with the other engine on the machine, Apple's GPU through the vendor's MPSGraph GEMM, in f32, the only precision it has.

Setting. Row-major $A \in \mathbb{R}^{m \times k}$, $B \in \mathbb{R}^{k \times n}$, $C = AB$, in f32 or f64. The machine is an Apple M4 Pro (two performance clusters of five cores, one efficiency cluster of four, 64 GB) running macOS 26; Accelerate's GEMM runs on the SME matrix units, one per cluster, at about 3.3 TFLOP/s in f32 and 0.85 in f64 on large square problems. Times are the best of five interleaved rounds (each round runs every method once; medians are within 2% unless stated). Errors are measured as defined above ("err64" in the tables); the tables of Section 6 also list "relerr", the same max-norm difference taken against Accelerate's product, which includes the GEMM's own error and is kept only for comparison with the earlier campaigns.

n, prec.	mixed plan	time (s)	error	best 4×4-only	time (s)	error
6912 f32	`R363+4/D48/G6`	0.141	$2.26\cdot10^{-6}$	`D48+4/D48/G6`	0.149	$2.94\cdot10^{-6}$
13824 f32	`R363+4/R363/G6`	0.907	$2.46\cdot10^{-6}$	`D48+4/D48/G6`	0.949	$4.35\cdot10^{-6}$
6912 f64	`R363+3/R633/G6`	0.451	$3.75\cdot10^{-15}$	`D48+4/D48/G6`	0.478	$5.60\cdot10^{-15}$
13824 f64	`R363+3/R336/G3`	3.296	$6.40\cdot10^{-15}$	`D48+4/D48/G6`	3.801	$8.28\cdot10^{-15}$

Table 1: Matched time: the $3\times3\times6$-family plan against the best plan from the 4×4 scheme alone (M4 Pro, square problems, best of three to five rounds, error against the exact product). Accelerate alone takes 0.195, 1.565, 0.773 and 6.223 s on these four with errors $3.85\cdot10^{-6}$, $6.40\cdot10^{-6}$, $6.69\cdot10^{-15}$ and $1.23\cdot10^{-14}$. All four mixed plans are the planner's choices under its default budget. The 13824 f64 row is from the budget campaign (`results/budget_campaign_2026-10-29.txt`), whose runs were 3% slower throughout (Accelerate 6.30 s), so both of its times are that run's; without a budget the planner takes `SW/R363+3/R633/G6` there, 1% faster with $1.25\cdot10^{-14}$, the GEMM's own error (Section 6).

	GPU (MPSGraph)		Accelerate		best mixed plan	
n, f32	time (s)	error	time (s)	error	time (s)	error
3456	0.013	$3.04\cdot10^{-6}$	0.025	$3.04\cdot10^{-6}$	0.021	$1.28\cdot10^{-6}$
6912	0.099	$3.85\cdot10^{-6}$	0.195	$3.85\cdot10^{-6}$	0.141	$2.26\cdot10^{-6}$
13824	0.946	$6.40\cdot10^{-6}$	1.565	$6.40\cdot10^{-6}$	0.907	$2.46\cdot10^{-6}$

Table 2: The GPU against the CPU in f32 (the GPU has no f64): the M4 Pro's 20-core GPU through MPSGraph's matrix multiplication on zero-copy shared buffers, wall clock; Accelerate's GEMM; and the best mixed plan (`R363/G6`, `R363+4/D48/G6`, `R363+4/R363/G6`). The GPU's product is bit for bit Accelerate's, so its error is the GEMM's (Section 6.1).

2 Fused levels and what they cost

2.1 Schemes as linear programs

A scheme $\langle g_1\times g_2\times g_3 : r\rangle$ is three matrices $L\in\mathbb{R}^{r\times g_1g_2}$, $R\in\mathbb{R}^{r\times g_2g_3}$, $P\in\mathbb{R}^{g_1g_3\times r}$ with

$$\operatorname{vec}(AB) \;=\; P\big((L\operatorname{vec}A)\circ(R\operatorname{vec}B)\big) \qquad \text{for all } A\in\mathbb{R}^{g_1\times g_2},\ B\in\mathbb{R}^{g_2\times g_3}, \tag{1}$$

$\circ$ the entrywise product and vec row-major. Applied to blocks, the same identity holds with A's g_1g_2 blocks of size $\frac{m}{g_1}\times\frac{k}{g_2}$, B's g_2g_3 blocks of $\frac{k}{g_2}\times\frac{n}{g_3}$, and C's g_1g_3 blocks of $\frac{m}{g_1}\times\frac{n}{g_3}$: the r products are $(\frac{m}{g_1}\times\frac{k}{g_2})(\frac{k}{g_2}\times\frac{n}{g_3})$ matrix products, computed by the plan below. A *level* is the application of (1) once; the matrices are realized as straight-line programs (PLinOpt's `.slp` syntax [6]), one per side, and the three programs are what a level executes:

- the L pass reads the g_1g_2 blocks of A and writes the r left factors S_i;
- the R pass reads the g_2g_3 blocks of B and writes the r right factors T_i;
- the products $M_i=S_iT_i$ run (GEMMs, or a level below);
- the P pass reads the r products and writes the g_1g_3 blocks of C.

level	scheme	saving	streams $L/R/P$	traffic (n^2)	amp_{eq}	γ_{eq}
`SW`	Strassen–Winograd $\langle 2\times 2\times 2 : 7\rangle$	0.875	11/11/11	8.25	10.66	19.8
`D48`	DPS accurate $\langle 4\times 4\times 4 : 48\rangle$, 262-op. point	0.750	64/64/64	12.00	2.828	5.657
`R336`	DPS accurate $\langle 3\times 3\times 6 : 40\rangle$	0.741	49/58/58	11.89	2.280	6.162
`R363`	the same, orientation $3\times 6\times 3$	0.741	58/58/49	11.89	2.453	5.125
`R633`	the same, orientation $6\times 3\times 3$	0.741	58/49/58	11.89	2.280	6.162
`T336`	ternary $\langle 3\times 3\times 6 : 42\rangle$ (Section 7)	0.778	51/60/60	12.33	6.407	11.07
`T363`	orientation $3\times 6\times 3$	0.778	60/60/51	12.33	6.166	9.737
`T633`	orientation $6\times 3\times 3$	0.778	60/51/60	12.33	8.061	22.84

Table 3: The levels: saving of products per level, blocks moved by each pass, pass traffic on a square problem in units of n^2 words, and the two accuracy measures of Section 3, normalized to the classical algorithm of the shape and raised to the power $\ln 0.75/\ln(\text{saving})$ so that they are comparable per 4×4-equivalent saving.

Each pass is *fused*: one loop over the element positions of a block, with the whole program evaluated on scalars in registers at each position. A pass therefore reads every input block once and writes every output block once, and the program's operation count costs only arithmetic, which the memory system hides.

2.2 The cost of a level

Write w for the word size and B for the streaming bandwidth the passes reach. A level on an $(m\times k)(k\times n)$ problem moves

$$W(m,k,n) \;=\; (g_1g_2+r)\,\frac{mk}{g_1g_2} \;+\; (g_2g_3+r)\,\frac{kn}{g_2g_3} \;+\; (g_1g_3+r)\,\frac{mn}{g_1g_3} \tag{2}$$

words through its three passes, that is $n^2\big(3+r(\frac{1}{g_1g_2}+\frac{1}{g_2g_3}+\frac{1}{g_1g_3})\big)$ on a square problem, and costs

$$T(m,k,n) \;=\; r\,T_{\text{below}}\Big(\frac{m}{g_1},\frac{k}{g_2},\frac{n}{g_3}\Big) \;+\; \frac{w\,W(m,k,n)}{B}. \tag{3}$$

Two numbers of a scheme decide the second term: its *traffic* W/n^2 and its *streams*, the number of blocks a pass reads and writes at once, g_1g_2+r, g_2g_3+r and g_1g_3+r. Beyond about 64 streams a pass loses bandwidth: a $\langle 5\times 5\times 5 : 93\rangle$ level's 118-stream passes ran at half the bandwidth of the 64-stream passes of the 4×4 scheme. Table 3 gives these numbers for the menu. The first term is decided by the *saving* $r/(g_1g_2g_3)$ and by the GEMM's rate on the products' shape and size: the f32 GEMM keeps 3.2–3.3 TFLOP/s down to 2048 and drops to 3.0 at 1024 and 2.6 at 512, where the f64 GEMM keeps its 0.81–0.85 TFLOP/s down to 1024, so f64 rewards depth and f32 does not.

2.3 What the implementation taught

The passes reach 120–135 GB/s against a peak of about 270. Two details were each worth a large factor. With 64 streams the compiler gave up vectorizing the fused loop (it could not rule out overlap among 64 pointers); a `vectorize(assume_safety)` pragma gave 4.3× on the passes. And scratch blocks of power-of-two size all start at the same offset modulo the cache's set span, so element c of all 64 streams fell into one cache set; padding each block by 1040 elements gave another 2.5×. Operation counts did not matter: our 201-operation program for the 4×4 scheme [19], DPS's 315-operation original and their 284-operation accurate program run at the same speed

as fused passes (totals within 1%; the 315-operation passes take 2–8% longer, about 0.3% of the total). The exception is a wasteful program: the direct sums of the $3 \times 3 \times 6$ scheme's R and P matrices cost 18 operations per output, and their passes ran at 0.092 s against 0.070 for the 4×4 level's at 13824 in f32 for 0.99× the words. PLinOpt's optimizer [6] brought the nine programs of the three orientations from 152, 344+384 and 366+384 operations (L, R, P of the $3 \times 3 \times 6$ orientation; additions plus scalings) to 59, 117+22 and 148+18, which cut the R-level passes by a quarter and the plans by 1–7% with the error unchanged.

2.4 Using the whole machine

Accelerate drives each performance cluster's SME unit from one thread, so its GEMM keeps two or three cores busy and leaves the rest idle. The r products of a level are independent, and a leaf `G`k runs them as k single-threaded GEMM streams taking products in turn: three streams beat Accelerate's own threading by 10% (f32) and 8% (f64) at 2048, more streams win for the smallest products (f32 at 1024, f64 at 512) and lose badly for large ones (six streams +26–34% in f64 at 1024). The calibration times each level's products with 0, 3, 4 and 6 streams and the planner chooses per plan: 6–14% off the best plans.

A streaming pass beside the SME streams slows them, but less than the pass costs run alone: at f64 blocks of 512 two pass threads slow six streams by 14% while the pass runs at 49 GB/s, about 35% of its stand-alone cost. So a level whose products are themselves levels over GEMM streams can *pipeline* them (written `X+p`, p pass threads): while the streams compute sub-problem q, the pass threads write sub-problem $q-1$'s result and the factors of $q+1$, the scratch below double-buffered. Pipelining takes 8–22% off the plans over six-stream leaves; one pass thread is too few, and sub-problems under 1 ms are not pipelined. The calibration times a pipeline of 2 and of 10 synthetic sub-problems per level, block size, stream count (3, 6) and pass threads (2, 3, 4): the slope is the steady time per sub-problem, the rest the fill and drain. NEON GEMMs on the idle cores were tried and not adopted: at most 1–3%, fragile, and the SME units lose most of what the vector units add.

3 The menu: which schemes can help

In a fused-pass library a plan already is a combination of sub-schemes, one per level on the current (m, k, n) sub-problem, so the question of mixing is the question of the menu. We scored every scheme on disk – AlphaEvolve's 15 shapes [14], the 29 rectangular schemes of Kauers and Wood [10] and the $5\times5\times6$ and $6\times6\times6$ schemes of Moosbauer, Poole and Kemper [12], both as distributed with the addition-reduction library of Mårtensson and Stankovski Wagner [13], PLinOpt's small and rectangular schemes including DPS's accurate variants [6], Smirnov's $\langle 3 \times 3 \times 6 : 40 \rangle$ [18], and Kronecker composites of the accurate pieces – by the quantities that decide a level's cost and error in (3):

- saving $r/(g_1 g_2 g_3)$, streams $g_1 g_2 + r$, $g_2 g_3 + r$, $g_1 g_3 + r$, and traffic $3 + r \sum 1/(g_i g_j)$;
- the amplification of the leaf error,

$$\text{amp} \;=\; \max_j \; \tfrac{1}{2} \sqrt{\textstyle\sum_i P_{ji}^2 \, \|L_i\|_2^2 \, \|R_i\|_2^2}, \tag{4}$$

  the root-sum-square growth of independent rounding errors in the r products through output j, normalized by the classical algorithm's $\frac{1}{2}\sqrt{g_2}$; and DPS's growth factor $\gamma_{\infty,2} = \max_j \sum_i |P_{ji}| \, \|L_i\|_2 \|R_i\|_2$ [1, 3], normalized by g_2.

scheme	saving	streams	traffic	amp_{eq}	γ_{eq}
$\langle 3\times3\times6:40\rangle$ DPS accurate (and $6\times3\times3$)	**0.741**	49/58/58	**11.89**	**2.280**	6.162
$\langle 3\times6\times3:40\rangle$ DPS accurate	0.741	58/58/49	11.89	2.453	**5.125**
$\langle 4\times4\times4:48\rangle$ accurate, 262-op. point	0.750	64/64/64	12.00	2.828	5.657
$\langle 3\times4\times6:54\rangle$ AlphaEvolve (±1 point)	0.750	66/78/72	12.75	4.637	6.784
$\langle 3\times4\times7:63\rangle$ rational	0.750	75/91/84	13.50	5.734	11.13
$\langle 5\times5\times5:93\rangle$ AlphaEvolve	0.744	118/118/118	14.16	8.560	17.46
$\langle 5\times5\times6:110\rangle$ Kemper	0.733	135/140/140	14.73	10.38	22.9
$\langle 6\times6\times6:153\rangle$ Moosbauer–Poole	0.708	189/189/189	15.75	9.229	18.3
$\langle 2\times4\times5:32\rangle$ AlphaEvolve	0.800	40/52/42	11.80	4.962	10.95
$\langle 2\times2\times2:7\rangle$ DPS accurate (in $\mathbb{Q}(\sqrt{3})$)	0.875	11/11/11	8.25	3.855	10.53
$\langle 2\times2\times2:7\rangle$ Strassen / Winograd	0.875	11/11/11	8.25	6.89 / 10.66	14.1 / 19.8
$\langle 3\times3\times3:23\rangle$ best of four	0.852	32/32/32	10.67	12.7	34.1
$\langle 3\times3\times6:40\rangle$ Smirnov (±1 point, same class)	0.741	49/58/58	11.89	9.754	26.1

Table 4: The menu screen (`matmul/levels/levels.py`): the schemes that could matter, at their published points. The accuracy columns are per 4×4-equivalent saving.

Schemes of different shapes save different fractions per level, so the normalized factors are raised to the power $\ln 0.75/\ln(\text{saving})$, the growth per 4×4-equivalent saving, and are then directly comparable with the 4×4 scheme's 2.828 and 5.657 (the columns amp_{eq} and γ_{eq} of Table 3). The complete table is `matmul/levels/levels.csv` [21]; Table 4 shows the candidates.

Three conclusions. Everything larger than 4×4 saves more per level but needs 110–225 streams per pass, where the passes run at half bandwidth, and has 8–100× the classical amplification at its published points. Kronecker products add nothing: a composite keeps the per-saving amplification of its pieces ($\langle3\times3\times6\rangle\otimes\langle3\times3\times6\rangle$ keeps 2.280, the accurate 2×2 squared keeps 3.855) and multiplies their stream counts, so it is never better than its two levels run one after the other and the stream explosion makes it worse. And the one scheme that beats the 4×4 scheme on every count but γ_{eq} is DPS's accurate $\langle3\times3\times6:40\rangle$: applied to a square problem, the $3\times3\times6$ orientation hands down $(n/3,n/3,n/6)$ products, the $3\times6\times3$ orientation on those gives $(n/9,n/18,n/18)$, and the $6\times3\times3$ orientation returns to the square $(n/54)^3$. The cycle keeps 0.741 of the products per level, moves $11.89\,n^2$ words, stays within 58 streams, and amplifies the leaf error by 2.28–2.45 per 4×4-equivalent saving against 2.83. Perminov's catalogue of 680 shapes up to $16\times16\times16$ [16] confirms the menu: only $\langle3\times3\times6:40\rangle$, $\langle4\times4\times4:48\rangle$ and $\langle3\times4\times6:54\rangle$ save a quarter of the products within 80 streams per pass, and the new shapes that save a quarter, $\langle3\times6\times8:108\rangle$ and $\langle4\times7\times9:187\rangle$, need 126–215 streams.

The paper cost model (3) with the GEMM at the lone 8192^3 f32 rate and the passes at the 4×4 level's bandwidth puts the cycle 1–3 points ahead of the 4×4 plans at every depth (`R336/G` 17.8% off the GEMM against `D48/G` 16.8%; `R336/R363/G` 13.9% against 10.8%), and the assumption behind it holds: Accelerate on the rectangular leaves at equal flops runs within 5% of the square GEMM after one level and 13% ahead after two, because the leaves are larger ($n/54$ against $n/64$ after three levels).

4 Accuracy

4.1 What a level does to the error

The rounding errors of the r leaf products pass through P into the outputs; the roundings of the passes themselves add to them. Taking the leaf errors as independent and proportional to the sizes

of the factors gives (4): a level multiplies the leaf error by amp, against $\frac{1}{2}\sqrt{g_2}$ for the classical algorithm on the same blocks. This root-sum-square measure, not the worst-case sum $\gamma_{\infty,2}$, is what the measurements follow: the 4×4 orbit's $\gamma_{\infty,2}$ minimum (21.41, in $\mathbb{Q}(\sqrt{2})$) has amp $= 3.14$ and measures no better than DPS's Table 1 point, while the 262-operation point, with amp $= 2\sqrt{2}$ the orbit's minimum and the smallest $\gamma_{\infty,2}$ among dyadic points, measures 0.75–0.77 of it [20].

Over its whole orbit under $GL_3 \times GL_3 \times GL_6$ the $\langle 3 \times 3 \times 6 : 40 \rangle$ class has amplification ratio at least 2.242 in every orientation (L-BFGS from random starts, `orbit_rect.py`); DPS's $3 \times 3 \times 6$ point is 5% above that floor and 5% above its $\gamma_{\infty,2}$ floor, their $3 \times 6 \times 3$ point 14% above (and 41% above that orientation's $\gamma_{\infty,2}$ floor of 3.90, far below the 4×4 class's 5.66). The ± 1 Smirnov point of the same class (amplification ratio 10.8) reaches the same floors, so the class carries them, not the point. Per 4×4-equivalent saving the floor is 2.17 against the 4×4 class's 2.83, which is its orbit's minimum and, as far as continuous search from many starts can tell, a local minimum of the whole variety of rank-48 decompositions. A better exact point for the $3 \times 6 \times 3$ orientation is worth a dyadic search.

4.2 The error model and its validation

A plan's error is well described by the GEMM's own error scale times one factor per fused level and a depth factor per level after the first, growing like $\sqrt{k}$ with the inner dimension,

$$\mathrm{err}(\mathrm{plan};\, m, k, n) \;=\; e_0 \sqrt{k/N_E}\; \rho^{L-1} \prod_{\text{levels } X} f_X, \tag{5}$$

with the order of the levels immaterial (measured: `SW` then `D48` and `D48` then `SW` within 2%) and streams and pipelines leaving the arithmetic unchanged. The calibration measures seven plans at $N_E = 2304$ (divisible by 16, 3 and 6) against the exact product – the f64 product of the f32 inputs, the compensated product of the f64 inputs: the GEMM alone, which gives e_0, one level of each scheme over the GEMM, which gives its factor, and `D48/D48`. In f32 the factors are `SW` 1.41, `D48` 0.62, `R336` 0.55, `R363` 0.45, `R633` 0.60; in f64 1.51, 0.72, 0.65, 0.47, 0.64. The accurate levels' factors are below one: their leaves accumulate k/g_2 products instead of k, and the GEMM's own accumulation error is the larger term. A second level does not get that again, so the factors do not simply multiply; the two-level plan measures the shortfall, a depth factor ρ (2.19 in f32, 1.53 in f64) by which the product of the factors is multiplied once per level after the first, ρ^{L-1} for L fused levels in (5). Against the exact errors of the campaigns' plans at 3456–13824 (Tables 5 and 6) the model predicts the plans without a Strassen level within 0.66–1.12 of the measurement (the under-predictions are at 13824 f64, where the GEMM's own error exceeds its $\sqrt{k}$ extrapolation by 1.3×) and over-predicts the plans with one by up to 1.6× in f32, the safe side; the growth over the GEMM, which the budget uses, is within 25%. Until this refit the factors were fit to differences from Accelerate's product, which include the GEMM's own error: they ranked the accurate levels wrongly and put every f64 plan above the GEMM, which is why the campaigns before the budget's introduction ran without one.

4.3 Programs and precision

For a fixed scheme the program still matters, through the roundings of the P pass: a first-order rounding model with exact second moments of every intermediate shows that DPS's 262-operation programs collect 15 roundings of RMS size 0.80 of the output per output where our 445-operation programs for the same point collect 36 of size 1.39, and among four programs of equal amplification the model's worst-output error orders the measured errors exactly at three and five levels [20]. The

lever is nearly exhausted for the 4×4 class: an 11% smaller modelled pass error was invisible at one to three levels and worth about 5% at depth. The precision lever remains: running the passes of the f32 path in f64 (the blocks stay f32, the arithmetic is f64, one rounding at the GEMM interface) costs $\leq 2\%$ below 16384 and cuts the error by 36% at five levels, nothing at one to three, where the leaf GEMMs' error dominates. It is a switch in the library (`FMM_F64PASS`), not the default.

5 The planner

A plan is a sequence of levels from the top, ending in a leaf: `G` (one GEMM, Accelerate's threading), `G`k (k GEMM streams), or the classical blocked GEMM `B2` on its own. The planner is a dynamic program over the first level:

$$\mathrm{cost}(s) \;=\; \min\Big(\mathrm{leaf}(s),\; \min_X \big[\mathrm{passes}_X(s) + r_X\,\mathrm{cost}_X(s/g_X)\big]\Big), \tag{6}$$

where $s = (m, k, n)$, s/g_X is the shape of X's products, and $\mathrm{cost}_X(t)$ is the cheaper of a GEMM leaf *priced inside level* X (with 0, 3, 4 or 6 streams) and a further level on t. Pricing the leaf inside the level matters: a GEMM shorter than about 1 ms takes 13–27% longer called back to back, as inside a plan, than called alone, and the first calibration, which timed single calls, picked a two-level plan at 8192 f64 that ran 12% slower than the right one. A level whose sub-problems are a level over GEMM streams may instead be pipelined (Section 2.4), priced from the calibrated steady time per sub-problem plus fill and drain.

The error budget. Every call takes a number g: `max_growth` in the C and Rust interfaces, `auto:`g in the benchmark. For each candidate plan the planner computes its predicted *error growth*, the factor $\rho^{L-1}\prod_X f_X$ of the error model (5): the plan's predicted error divided by the classical GEMM's predicted error on the same problem, so that a growth of 1 means as accurate as the GEMM, 0.5 twice as accurate, 2 twice the error. With $g > 0$ the planner returns the fastest plan whose growth is at most g: $g = 1$ asks for a plan no less accurate than the GEMM it replaces, $g < 1$ for a more accurate one, and $g > 1$ admits plans up to g times the GEMM's error, which is where the plans with a Strassen level and the deep plans live. $g = 0$ means the default budget, 1; a negative g means no budget, the fastest plan whatever its error (`auto:none` in the benchmark), which is how the campaigns before the budget's introduction ran. So no budget can return a less accurate plan than the default: at 13824 f64 it returns `SW/R363+3/R633/G6`, whose exact error is 1.02× the GEMM's, where the default returns `R363+3/R336/G3` at 0.52× in the same time (Table 6). The growth is the model's estimate, good to about 25% (Section 4), so a budget is a target rather than a guarantee, and near a boundary the choice is not robust; the Strassen-topped plans sit at 1.0–1.1 and the plans without one below 0.85, so the default separates the two families with the margin the model has.

Calibration. Everything the model needs is measured once per machine and precision and cached (format 5, `calib5_f32.txt`, `calib5_f64.txt`, keyed by the machine, the padding, the file format and the programs the kernels came from): the lone GEMM and the blocked GEMM at each size; for each level and product shape its three passes and the GEMM time per product at 0, 3, 4 and 6 streams, measured together by running the level; the pipelines; and the seven error-model plans at $N_E = 2304$, against the exact product. For n up to 13824 this is 408 entries and 13 minutes per precision. Square shapes interpolate (log-log between the measured sizes); rectangular shapes do not, and are either calibrated for the shape (118 entries, a few seconds) or priced by *shape estimates*

from the square calibration: GEMM-like entries at the square size of equal flops, $\sqrt[3]{mkn}$ (Accelerate runs rectangular shapes within 5% of the square rate), and a level's passes at the square size of equal traffic,

$$n_{\text{eq}} = \sqrt{\frac{(g_1g_2 + r)\,mk + (g_2g_3 + r)\,kn + (g_1g_3 + r)\,mn}{g_1g_2 + g_2g_3 + g_1g_3 + 3r}}, \tag{7}$$

the size at which a square problem moves the same words through the level's passes (2). The estimates predicted every measured rectangular case within 2% (Table 7; 5% for the one plan the default budget changed).

Sizes and shapes the levels do not divide. A plan runs at the smallest shape its levels divide: every dimension is rounded up to the same multiple of 2^a3^b ($a \le 10$, $b \le 3$; 44 candidates), the operands copied into zeroed matrices and the result copied back, priced at 2× the words at 80 GB/s. The planner takes the cheapest padded candidate, so an n that 3 does not divide gets a 4×4 plan unless padding to a multiple of 54 pays. Planning a new shape costs 0.07 s in C and 0.010 s in Rust and is cached for 64 shapes.

Validation. Predictions are within 0.3–3% of the measured times at $n \ge 8192$ in both precisions throughout the development (first with the 4×4 menu, then with streams, pipelines and the $3\times3\times6$ levels), and the planner's choice was the fastest plan measured, or within 1% of it, at every size and precision tried, with two documented exceptions traced to noisy calibration entries (a pipeline slope from two and four sub-problems; a 4096 f32 run 15% slow throughout). The classical variants themselves drift between runs (13824 f64: 6.2–6.7 s), so speedups carry about ±5%.

6 Results

Table 5 compares, at two sizes and in both precisions, the single levels, the best plan the 4×4 menu offered, and the planner's mixed plan. What the table says:

- As a single level the $3 \times 3 \times 6$ scheme ties the 4×4 scheme in time at 13824 (in f32 at 6912 it is 3–6% slower, because the passes are a larger share there and its L pass moves more) and is more accurate in every orientation: `R336` by 7–19%, `R363` by 34–41% in both precisions (1.76 against $2.81 \cdot 10^{-6}$ at 6912 f32, 4.97 against $7.80 \cdot 10^{-15}$ at 13824 f64). The planner knows the pass costs and uses the orientation whose cheapest pass faces the largest operand.

- At matched time the mixed plans are the more accurate ones. In each of the four cases the $3 \times 3 \times 6$-family plan that runs in the time of the best 4×4-only plan, `D48+4/D48/G6`, has 23–43% less error against the exact product and is 4–6% faster: `R363+4/D48/G6` at 6912 f32 (2.26 against $2.94 \cdot 10^{-6}$), `R363+4/R363/G6` at 13824 f32 (2.46 against $4.35 \cdot 10^{-6}$), `R363+3/R633/G6` at 6912 f64 (3.75 against $5.60 \cdot 10^{-15}$) and `R363+3/D48/G6` at 13824 f64 (5.81 against $8.28 \cdot 10^{-15}$). These plans, at 1.4–1.8× over the GEMM, also have less error than a single 4×4 level at 1.25× (`D48/G`). All four are the planner's choices under its default budget (at 13824 f64 the default's own pick, `R363+3/R336/G3`, is faster still: the next item), and its top six at 13824 f32 are all $3 \times 3 \times 6$-family plans.

- Without a budget the planner's choice at 13824 f64 is `SW/R363+3/R633/G6`: a Strassen–Winograd level on top buys 1.91× against 1.68 for the best 4×4 plan at the price of 51% more error ($1.25 \cdot 10^{-14}$ against $8.28 \cdot 10^{-15}$; the GEMM's own is $1.23 \cdot 10^{-14}$). The default budget

rejects it (predicted growth 1.07) and takes `R363+3/R336/G3`, a plan no earlier campaign had measured: 3.30 s against the Strassen plan's 3.27 in the same run, with error $6.40 \cdot 10^{-15}$, half the GEMM's (Table 6). At 3456 f64 the same happens: `R363/G4` at 0.072 s and 0.58× the GEMM's error replaces `SW+3/R363/G6` at 0.071 s and 0.97×. The Strassen plans remain available under an explicit budget of 2 or none.

- Every plan without a Strassen level has less error than the GEMM's own in both precisions (2.26 against $3.85 \cdot 10^{-6}$ at 6912 f32, 2.46 against $6.40 \cdot 10^{-6}$ at 13824 f32, 3.75 against $6.69 \cdot 10^{-15}$ at 6912 f64, 5.81 against $1.23 \cdot 10^{-14}$ at 13824 f64): a fast multiplication that is also the more accurate one (Section 4 says why). The difference from Accelerate's product, the only f64 error the earlier campaigns recorded, hides all of this: it mixes the GEMM's error with the plan's.
- The three-orientation cycle alone (`R336/R363/R633/G`) does not pay at these sizes: its leaves of $n/54$ are too small for the GEMM. Depth pays through the pipelined two-level plans.

n, prec.	plan	budget	time (s)	growth over the GEMM		error
				predicted	measured	
3456 f64	`R363/G4`	default	0.072	0.47	0.58	$2.76 \cdot 10^{-15}$
	`SW+3/R363/G6`	none	0.071	1.09	0.97	$4.62 \cdot 10^{-15}$
13824 f64	`R363+3/R336/G3`	default	3.296	0.47	0.52	$6.40 \cdot 10^{-15}$
	`SW/R363+3/R633/G6`	none	3.267	1.07	1.02	$1.25 \cdot 10^{-14}$
3456 f32	`R363/G6`	both	0.021	0.45	0.42	$1.28 \cdot 10^{-6}$
6912 f32	`R363+4/D48/G6`	both	0.141	0.60	0.59	$2.26 \cdot 10^{-6}$
6912 f64	`R363+3/R633/G6`	both	0.451	0.46	0.56	$3.75 \cdot 10^{-15}$
13824 f32	`R363+4/R363/G6`	both	0.916	0.44	0.38	$2.46 \cdot 10^{-6}$

Table 6: The planner's choices with the default budget (1: no less accurate than the GEMM) and without one (`results/budget_campaign_2026-10-29.txt`; best of three to five rounds, errors against the exact product). "Growth" is the plan's error over the GEMM's on the same problem, as the model predicts it and as measured; "both" means the two budgets choose the same plan.

The planner's choices by size, from the calibration at 13824 and under the default budget: f32 `G` at 1728, `R363/G6` at 3456, `R363+4/D48/G6` at 6912, `R363+4/R363/G6` at 13824; f64 `R363/G6` at 1728, `R363/G4` at 3456, `R363+3/R633/G6` at 6912, `R363+3/R336/G3` at 13824 (without a budget, `SW+3/R363/G6` at 3456 and `SW/R363+3/R633/G6` at 13824). Sizes that 3 does not divide keep their 4×4 plans (`D48+4/D48/G6` at 8192 f32, 1.40×; `D48+3/D48/G6` at 9000 f64 padded to 9008, 1.69×).

Rectangular problems (Table 7) go through the same planner over (m, k, n): a level applies when its grid divides the current shape, so a $6912 \times 6912 \times 3456$ product gets a 4×4 level whose products are $1728 \times 1728 \times 864$ and then a $3 \times 6 \times 3$ level in f64. The shape estimates are within 2% of the measurements wherever a level pays (5% for the f32 plan the default budget changed), and where none does the planner correctly returns the GEMM (the first version of the estimates had none, and padded $1000 \times 3000 \times 2000$ to 3072^3, 30 times slower than Accelerate).

6.1 The GPU as the alternative

The other engine on the machine is the 20-core GPU. We measured Apple's own GEMM paths on it (`gpu_gemm.m`, Section 11 for the names): MPSGraph's matrix multiplication, the faster of

the two, and the older `MPSMatrixMultiplication`, within 3% of it at 13824 and 10% slower at 3456; in f32, since Metal, MPS and MPSGraph have no 64-bit floating type. The inputs are the benchmark's, wrapped as shared Metal buffers without a copy (the memory is unified), the time is the wall clock from encoding to completion, five rounds interleaved with Accelerate's GEMM on the same inputs, the error against the f64 product. Neither MLX nor PyTorch was installed, so these are the vendor's GEMMs, as Accelerate is on the CPU side.

What Table 8 says:

- The GPU's f32 product is bit for bit Accelerate's at every size. Both accumulate each output in increasing k with fused multiply-adds and neither splits the inner dimension, so both equal the sequential FMA chain (64 sampled outputs recomputed as that chain agree with both), and the GPU's error is the GEMM's. The mixed plans have 1.7–2.6× less error than either.
- In time the GPU is 1.6× ahead of the best plan at 3456 and 1.4× at 6912, and tied at 13824, where its throughput falls to 5.6 TFLOP/s from 6.6 at 6912 and the CPU plan's 0.907 s edges its 0.946.
- The GPU's f16-input path (f32 result) is only 14% faster than its f32 at 13824, and its error, $2.5\text{–}2.8 \cdot 10^{-4}$, is the rounding of the inputs to 11 bits.
- In f64 the GPU does not compete, and the plans' 1.71–1.91× over Accelerate's dgemm stands alone.
- The two engines share the package's power and thermal budget: Accelerate's sgemm interleaved with GPU runs took 1.81–1.88 s at 13824 against 1.565 alone. A hybrid that keeps both busy has to reckon with that; by estimate a $3 \times 3 \times 6$ level over GPU leaves at 13824 f32 would run in about 0.77 s (40/54 of the GPU's 0.946 plus 0.07 s of CPU passes), 1.2× over the GPU alone, if the leaf products keep the GPU's throughput. Not measured.

7 The ternary trial: cheap passes against accurate ones

Perminov's catalogue [16, 17] lists, beside the ranks, the schemes with coefficients in $\{-1, 0, 1\}$ whose additive complexity his parallel common-subexpression search reduced; for $3 \times 3 \times 6$ it has four rank-42 ternary schemes reduced to 139, 140, 145 and 150 additions for the three linear forms together (file `3x3x6_m42_cr139_cn287_ZT_reduced.json` and its siblings; the naive counts are 287–369). Against DPS's accurate rank-40 scheme, whose nine optimized programs take 374, 383 and 353 operations per orientation, this is the opposite design: two products more (saving 0.778 against 0.741), two more streams per pass (51/60/60), and a third of the pass arithmetic. Is the arithmetic worth anything once the passes are fused, and what does the accuracy cost?

The scheme. The reduced JSON is a straight-line program already (fresh variables for the common subexpressions, then the forms); each of the four schemes was expanded to its matrices and checked exactly against the Brent equations. The 139-addition scheme is the most accurate of the four by far (amplification ratio 5.07 in the $3 \times 3 \times 6$ orientation against 7.6–9.5 for the others), so it is the one tried; Table 3 gives its three orientations. Its programs are Perminov's forms for the three maps the $3 \times 3 \times 6$ orientation needs (L: 33 additions, R: 39, P: 67), and for the other orientations the same forms re-indexed where an orientation reuses a map and transposed by PLinOpt's `transpozer` where it needs the transposed map ($3 \times 6 \times 3$: 39, 43+6 negations, 66; $6 \times 3 \times 3$: 43+6, 33, 63), all nine checked by `SLPchecker` against the matrices. PLinOpt's optimizer (the fork's `-s`

`1 -Q -O 500`) on the same nine matrices finds 34, 41, 71, 41, 45, 67, 45, 34, 66 additions: one to four more than Perminov's reduction on every matrix, in 0.2 s each. The levels `T336`, `T363`, `T633` went into the C library beside the R levels, outside the planner's menu, and every plan containing them computes the product (3456, both precisions, including pipelined plans).

Measured. Table 9 answers both questions. The pass arithmetic is worth nothing: the T levels' passes take exactly the R levels' times (at 13824 f32, $L/R/P$ 0.036/0.020/0.017 s against 0.034/0.019/0.017; at 6912, 0.009/0.005/0.005 for both), because a fused pass is bound by the blocks it moves and the T level moves two more per pass. The two extra products cost what the rank says: the GEMM part of a single level is 5–7% longer, a single T level is 4–5% slower than its R counterpart at every size and precision (the one tie, `T363/G` at 6912 f32, is within the run's noise), a two-level T plan 8–10% slower, and the three-orientation cycle 13–16% slower. The accuracy cost is the amplification: 2.2–3.5× the R level's error as a single level, 3.7–4× in two-level plans and 6.5–7× in the cycle, in line with the ratio of the amplification factors ($5.07/2.36 = 2.1$ for the $3 \times 3 \times 6$ orientation, the measured 2.2–3.5 including the passes' own roundings). In f32 every T plan's error is above the GEMM's own (`T336/G` $1.28 \cdot 10^{-5}$ against $6.40 \cdot 10^{-6}$ at 13824) where every R plan's is below it. The planner's menu was left as it was: a level that is slower and less accurate than one already there on every plan cannot be chosen.

What it shows. The trial is the cleanest confirmation of the cost model's premise. Two schemes of the same shape, with the same pass structure, differ in rank by two and in pass arithmetic by a factor of 2.7; the measured difference is the rank's, to the percent, and none of the arithmetic's. For a fused-pass library the objectives of a scheme search are rank and amplification, in that order, and additive complexity is not among them. The caveat is the one stated in Section 2: arithmetic shows when a program is wasteful enough to make a pass compute-bound, as the 18-operation-per-output direct sums were; Perminov's 139 additions and DPS's optimized programs are both well inside the bandwidth-bound regime. Whether the class of the ternary scheme contains an accurate point is a separate question, and the answer is no: over $GL_3 \times GL_3 \times GL_6$ the class's amplification ratio is at least 2.825 in the $3 \times 3 \times 6$ orientation and 2.831 in $3 \times 6 \times 3$ (eight starts each), the 4×4 class's $2\sqrt{2}$ and 26% above the accurate $\langle 3 \times 3 \times 6 : 40 \rangle$ class's 2.242; the 139-addition point sits 1.8× above its own floor, and its $\gamma_{\infty,2}$ floors (5.12 and 4.82) are likewise above the accurate class's (6.37 and 3.90) in the orientation that matters. Even at its best point the class would match the 4×4 scheme's accuracy with two products more than the accurate $3 \times 3 \times 6$ scheme.

8 Two implementations

The library is C (`fmm.h`, `libfmm.a`: `fmm_sgemm`, `fmm_dgemm` and their `_mkn` forms for an $m \times k$ times $k \times n$ product with separate leading dimensions, `fmm_calibrate`, `fmm_plan`, `fmm_release`) over Accelerate, with the kernels generated from the schemes' programs by `gen_kernels.py`: every program in PLinOpt's syntax becomes one fused loop in f32 and in f64, plus an f64-arithmetic variant for f32 blocks. Plans are made once per (shape, precision, budget) and kept; the workspace is kept between calls; a call outside the calibrated range or without a calibration runs the classical GEMM, so a call is always correct. The benchmark driver includes the library, so both run the same code, and every plan prints its predicted time next to the measured one.

The Rust port (crate `fmm`, `rust/`) is the same library module for module – plans and shapes, the calibration in the C file format so that either library reads the other's cache, the planner with pipelines, budget and padding, the executor on a rayon pool, the entry points – with the fused

passes as safe Rust emitted by the same generator (each slice a separate parameter, cut to the row length first: the compiler then vectorizes as `restrict` and the pragma let it in C). The only unsafe code is the three FFI declarations for Accelerate behind safe wrappers that check the slices. Cross-checked against the C library on the shared calibration, 26 cases (square sizes from 1000 to 9000, a leading dimension above n, an error budget, four rectangular shapes, both precisions) give the same plans, the same predicted times to the digit and results bit for bit, before and after the error model's refit and the default budget, which both ports carry. Speeds match at large sizes (f64 8192: 0.801 against 0.815 s; f32 $6912 \times 6912 \times 3456$: 0.0789 against 0.0796); the Rust passes on blocks under about 600 are 1.2–2× slower (dispatch and per-chunk allocation), which only shows when Rust calibrates for itself. Two traps found on the way: a zeroed `Vec` gets its pages on first write, so timed buffers must be written first; and a lone multi-threaded GEMM issued right after GEMM streams on the rayon pool runs 15–25% slower and stays so, which the leaves' timing avoids by running where the library runs a classical plan.

9 Discussion

The mixed approach rests on two measurements that contradict the usual accounting. A fused level's cost is the blocks it moves, so the right measure of a scheme is its saving per block moved and its stream count, not its rank or the length of its programs; and the accuracy of a level is set by the point in its orbit and, at depth, by the roundings of its P program, not by the operation count. Under these measures the menu is short: two accurate schemes and Strassen–Winograd for the top of deep f64 plans. A scheme enters the menu only if a plan containing it beats every plan without it on this machine, and the planner decides that per shape and precision from measurements, which is why the library can take a new level (or a new machine) without any change but a calibration. Its default objective is time at the GEMM's own accuracy or better; the Strassen plans, the only ones that breach it, are there for whoever asks.

The limits are also measured. The GEMM is 85–95% of every plan, so no menu can take more than the products it saves, and below $n \approx 3000$ nothing pays in f32 (1728 in f64). A level's passes must not fall behind the streams it feeds: six streams of 1024-blocks in f64 are the most sensitive to pass traffic, and pipelining gains nothing there. Shapes above 4×4 cannot help because of the stream count, whatever their rank, and Kronecker composites because they inherit their pieces' amplification and multiply their streams. And the ternary trial (Section 7) closes the remaining door on the cheap side: when the passes are already bandwidth-bound, a program with a third of the additions buys nothing, while two extra products and 2.8× the amplification cost speed and accuracy at every size.

What is next. A dyadic point of the $3 \times 6 \times 3$ orientation closer to its class's amplification floor (DPS's is 14% above it) would improve the plans that use it; the tools exist from the 4×4 search. Fusing the L and R passes into GEMM operand packing, the "Strassen reloaded" design, needs an open GEMM kernel; on this machine the vendor's SME kernels are closed, so it waits for another machine (x86-64 with BLIS, or the Apple GPU). The planner's 0.07 s per new shape in C should be a millisecond. The error model's depth factor is one number for all levels; a factor per level, or a model of the leaves' accumulation length, would tighten its 25%. The library takes $C = AB$ only: α, β and transposes are a day's work. And the Rust passes on small blocks should be brought to the C library's speed.

10 Commercial value, in software and in hardware

The accuracy gain by itself has little commercial value; it is what makes the speed gain sellable, and that combination has a real if narrow market. This section says where, and whether the design could be built into a GPU or a TPU.

Who would use it. Large dense products with a large inner dimension, in f32 or f64, whose accuracy is a concern, occur in scientific and engineering computing: Gram and covariance matrices over many samples, subspace orthogonalization in electronic-structure codes, tensor contractions in quantum chemistry, spectral transforms in weather and climate models, dense boundary-element and imaging matrices. They are GEMM-dominated, mostly f64, and their inner dimensions run to the thousands, which is where the plans pay. Two large markets are excluded. Machine learning runs bf16 or fp8 inputs with f32 accumulation, so its error is the input quantization, not the accumulation; a more accurate f32 or f64 product is worth nothing there, and that is the market that drives GEMM hardware. And the dense factorizations of LAPACK and the HPL benchmark call GEMM as rank-k updates with k of a few hundred, where no level applies (Section 5), so the most visible dense-linear-algebra workloads gain nothing.

What the result is worth. Not the last digit of a Gram matrix: the value is speed at no accuracy cost. Fast matrix multiplication has been known since 1969 and no production BLAS ships it; the standard objection is Higham's [7], that the Strassen-type algorithms lose accuracy. The mixed plans are 1.7–1.9× faster than the vendor's GEMM with less error than it, in both precisions, under a default that guarantees no less accuracy than the GEMM. That answers the objection in a form a library could adopt, and an HPC centre would take 40% of its dense-contraction time back for free. The realistic paths, in decreasing order of likelihood: adoption inside a vendor or open BLAS as a large-size path (the fused design needs an open kernel, so BLIS is the natural first target), with reputation rather than revenue as the return; consulting or a licensed library for codes with large contractions, a small market whose customers exist and pay; and nothing proprietary, since the schemes are published [3, 4] and the engineering is reproducible from this report. Two trends limit all of it. Dense f64 work is moving to GPUs, where accuracy is already sold in the other direction, f64 products emulated on low-precision tensor cores by the Ozaki scheme [15]; and on this machine the GPU already matches the f32 plan at 13824 (Section 6.1), so the clean win is f64, the smallest market of all. A port to an x86-64 server part with BLIS, measured, is what commercial traction would need.

In hardware. The design could speed up a GPU or a TPU in principle, but the right place for it is not the multiplier array. A matrix unit is compute-bound on large products (cuBLAS reaches 70–80% of peak), so a fused level of the accurate 4×4 scheme buys 25% of the array's cycles, a $3 \times 3 \times 6$ level 26%, and two levels about 45%, with passes of $O(n^2)$ work against the products' $O(n^3)$ – once the blocks are large. That condition rules the design out of the tensor core itself: for a 16×16 register tile the level's additions are comparable to its multiplies and the amplification is paid for nothing. The level must sit where blocks of hundreds of rows are resident, which on a GPU is shared memory and L2 and on a TPU the vector unit and its SRAM beside the systolic array; so it is a kernel and compiler feature with modest hardware help (wider adders on the operand path, staging SRAM), not a new multiply-accumulate design. The software form exists: the GPU kernels of Huang, Yu and van de Geijn [9] fuse a Strassen level's additions into the operand packing and gain 20–30% at large sizes, and Winograd's minimal convolution [11], the

same idea for convolutions, ships in several neural accelerators; vendors know the trick and have not applied it to GEMM. Precision decides whether it is worth building. For f64 and f32 units the answer is yes: an f64 multiplier is the most expensive arithmetic block on the chip, so trading a quarter of them per level for additions is an area and energy lever as well as a time one, and with the accurate schemes the level is more accurate than the plain product. For bf16 and fp8 units the answer is no: the operand combinations are not representable in the input format, so the L and R passes add a rounding per combination to inputs that already carry a 0.4% or 6% quantization error, and the scheme's amplification then costs more than the 25% saves; nobody in that market buys accuracy, and its throughput is better bought with narrower formats and more multipliers. Since that market pays for the silicon, the idea stays unbuilt. What an HPC-class part would gain is 20–45% more effective f64 or f32 GEMM throughput per watt at large sizes with no accuracy loss, for a kernel change and some staging logic; on this machine, where the GPU and the CPU plan tie at 13824 f32 and one $3 \times 3 \times 6$ level over the GPU's product is estimated at 1.2× (Section 6.1), fused passes and two levels would restore the GPU's lead at about 1.6×. What this report changes is not the arithmetic saving, known for decades, but the evidence that it need not cost accuracy, which is the argument a hardware team would need before spending area on it.

11 Glossary: the machine, its software, and the report's terms

The report uses Apple's names for the hardware and the libraries, and a few of Arm's. In one place, followed by the report's own terms and abbreviations:

Apple silicon, M4 Pro. Apple's system on a chip for Macs. The M4 Pro of this report has 14 CPU cores in three clusters (two of five performance cores, one of four efficiency cores), a 20-core GPU, and 64 GB of memory that all of them share. A cluster is Apple's group of cores around one level-2 cache; each performance cluster of the M4 also has one matrix unit.

Performance and efficiency cores. The two kinds of CPU core. The performance cores run the passes and the GEMM threads here; the efficiency cores are slower and share a smaller cache, and the thread counts of Section 2.4 are chosen so that timed work stays off them.

SME, Scalable Matrix Extension. Arm's instruction set extension for matrix arithmetic, implemented in the M4 as a matrix coprocessor with a large tile register and an outer-product instruction that multiplies a column of A by a row of B and accumulates into the tile. The M4 Pro has one SME unit per performance cluster, shared by that cluster's cores; it succeeds Apple's earlier undocumented AMX unit. Accelerate's GEMM runs on it: a GEMM call keeps about two cores busy and reaches 3.3 TFLOP/s in f32, far above what the cores' vector units could give, with the GPU's utilization counter at zero. It accumulates in the order of the sequential FMA chain, which is why its result equals the GPU's bit for bit. Its kernels are Apple's and closed, which keeps the "Strassen reloaded" design of fusing the passes into the GEMM's operand packing off this machine.

NEON. Arm's 128-bit SIMD vector instruction set, on every core. GEMMs on the idle cores with it were tried and not adopted (Section 2.4).

Accelerate. Apple's numerical library: BLAS and LAPACK, vDSP and more, with the kernels for Apple silicon. Its BLAS is the vendor GEMM of this report, called through the C interface `cblas_sgemm` and `cblas_dgemm`; the threading inside a call is Accelerate's own, and the library's streams are several single-threaded calls side by side.

BLAS, GEMM, sgemm, dgemm. The Basic Linear Algebra Subprograms interface; GEMM is its general matrix multiply $C \leftarrow \alpha AB + \beta C$, sgemm in single precision (f32), dgemm in double (f64). "The GEMM" in this report is Accelerate's.

FMA, fused multiply-add. The instruction computing $a \cdot b + c$ with a single rounding; every modern GEMM is built from it. One output accumulated as $c \leftarrow \mathrm{fma}(a_k, b_k, c)$ for k in order is the sequential FMA chain that both Accelerate's SME kernels and Apple's GPU kernels compute.

f32, f64, f16, bf16. IEEE single and double precision (24- and 53-bit significands), IEEE half precision (11 bits) and bfloat16 (8 bits). The CPU paths of this report are f32 and f64; Apple's GPU has f32, f16 and bf16 and no f64.

GPU, unified memory. The M4 Pro's 20-core graphics processor, programmed through Metal. It reads the same memory as the CPU, so a CPU array is handed to it without a copy by wrapping it as a shared Metal buffer; the GPU numbers of Section 6.1 include no transfers.

Metal. Apple's GPU programming interface and shading language, the counterpart of CUDA or Vulkan: devices, command queues, command buffers, compute kernels, buffers.

MPS, Metal Performance Shaders. Apple's library of ready GPU kernels on top of Metal, among them `MPSMatrixMultiplication`, a GEMM on Metal buffers.

MPSGraph. Metal Performance Shaders Graph: Apple's framework for tensor computation on the GPU, in which a graph of operations (here two placeholders and one matrix multiplication) is compiled to Metal kernels and run. It is what Core ML and PyTorch's Apple backend use underneath; its GEMM is the faster of Apple's two.

Grand Central Dispatch. Apple's thread-pool interface (libdispatch); `dispatch_apply` runs a block over an index range on the pool. The library uses both it and POSIX threads; the benchmark's compensated reference product runs on the pool through `dispatch_apply`.

Apple clang, macOS 26. The compiler and operating system of the measurements. The C library is C11 with Accelerate; the GPU tool is Objective-C, because the GPU frameworks have no C interface.

TFLOP/s. 10^{12} floating-point operations per second, counting $2mkn$ for a product whatever the algorithm, so a plan's rate is the classical count divided by its time.

GEMM. See above: the general matrix multiply of BLAS, here Accelerate's, the leaf of every plan and the baseline of every speedup.

Scheme $\langle g_1 \times g_2 \times g_3 : r \rangle$, rank. A bilinear algorithm that multiplies a $g_1 \times g_2$ by a $g_2 \times g_3$ block matrix with r block products instead of $g_1 g_2 g_3$; r is its rank. Its three matrices L, R, P are Section 2's.

Level, pass, plan, leaf. A level is one application of a scheme to the current problem, executed as three fused passes (L, R, P) around its products; a plan is the sequence of levels from the top down to the leaf, the GEMM; the notation is in Section 1.

`D48`, `R336`, `R363`, `R633`, `SW`, `T336`, `T363`, `T633`, `X`. The levels of the menu (Table 3): the accurate 4×4 scheme, the accurate $3 \times 3 \times 6$ scheme in its three orientations, Strassen–Winograd, and the ternary $3 \times 3 \times 6$ scheme of Section 7; `X` in Table 9 stands for `R` or `T`.

`G`, `G`k, `B2`, `+p`. The leaves and the pipeline mark: `G` is one Accelerate GEMM with its own threading, `G`k the products as k single-threaded GEMM streams, `B2` the classical 2×2 blocked product as a level (eight products, the calibration's reference); `+p` after a level runs its passes on p threads overlapped with the products below.

Streams, pipelines. Section 2.4: several single-threaded GEMM calls side by side, and a level's passes overlapped with the products of the level below.

err64, relerr. The max-norm relative error against the exact product (Section 1), and the same quantity against Accelerate's product, which includes the GEMM's own error and is kept only for comparison with the earlier campaigns.

Error growth, `max_growth`, `auto:`g. A plan's predicted error divided by the GEMM's, the product of the model's per-level factors f_X; the budget g a call sets on it: 0 the default of 1, a negative value no budget (Section 5).

amp, $\gamma_{\infty,2}$, N_E, e_0. The amplification measures of a scheme (root-sum-square and worst-case, Section 4); the size at which the error model is calibrated (2304) and its classical error scale there.

Kronecker composite. The scheme obtained as the Kronecker product of two schemes (one level of each, fused), screened out in Section 3.

DPS. Dumas, Pernet and Sedoglavic, the authors of the accurate schemes and of PLinOpt [1, 2, 3, 4]; DPSST adds Stapleton and Tichavský [5].

PLinOpt, SLP, SLPchecker. DPS's library of tools for linear programs [6]: a straight-line program (SLP) is a list of assignments computing a linear map; PLinOpt's optimizer shortens one, its transpozer transposes one, and SLPchecker verifies one against its matrix, which every program in this report passed.

Orbit, GL_n. The schemes equivalent to a given one under the general linear group GL_n of invertible $n \times n$ matrices acting on the three sides; the orbit floors of Section 4 are minima over it, found by L-BFGS, a quasi-Newton minimizer.

RMS, IEEE 754, SIMD. Root mean square; the floating-point standard (f32, f64, f16); single instruction, multiple data (vector instructions).

POSIX threads, FFI, JSON, C11. The standard thread interface the C library uses; the foreign-function interface through which Rust calls Accelerate; the text format of Perminov's catalogue; the 2011 C standard.

BLIS, CUDA, Vulkan, Core ML, PyTorch, MLX. Named for comparison only: an open BLAS framework (for the x86-64 machine the design of Section 9 waits for), Nvidia's and Khronos's GPU interfaces, Apple's and Meta's machine-learning frameworks, and Apple's array framework.

GB/s, TFLOP/s. Gigabytes per second of memory traffic; 10^{12} floating-point operations per second as defined above.

12 Data and code

Everything is in the repository [21], directory `matmul/fastmm/`: the library (`fmm.c`, `fmm_impl.h`, `fmm.h`), the kernel generator (`gen_kernels.py`), the programs of the $3 \times 3 \times 6$ levels (`schemes/opt/r*`, PLinOpt-optimized; `schemes/r*`, the direct sums) and of the ternary levels (`schemes/t*`), the Rust crate (`rust/`), the benchmark and the test (`bench_fmm.c`, `test_fmm.c`), the calibrations (`results/calib5_f32.txt`, `calib5_f64.txt`) and the raw results of every run quoted here: `results/r336_campaign_2026-10-28.txt`, `r336_campaign2_2026-10-28.txt` (Table 5), `r336_opt_vs_direct_2026-10-28.txt`, `rect_api_2026-10-28.txt` (Table 7), `t42_campaign_2026-10-28.txt` (Table 9), `f64exact_2026-10-28.txt` (the f64 err64 column of Table 5), `gpu_gemm_2026-10-28.txt` (Table 8; the tool is `gpu_gemm.m`, built by `make build/gpu_gemm`, the script `run_gpu.sh`), `budget_campaign_2026-10-29.txt` (Table 6; `run_budget.sh`), and the earlier campaigns of October 24–26 (streams, pipelines, accuracy, the 4×4 programs). The menu screen is `matmul/levels/` (`levels.py`, `levels.csv`, `orbit_rect.py`, `cost_rect.py`, README). The accuracy study of the 4×4 class is `doc/matmul_4x4_rank48_accurate262.pdf` [20]. The running log of the track, with every measurement in the order it was made, is `doc/fastmm_plan.md`.

References

[1] J.-G. Dumas, C. Pernet, A. Sedoglavic, *Strassen's algorithm is not optimally accurate*, Proc. ISSAC 2024, 254–263.

[2] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *non-complex multiplications*, arXiv:2506.13242 (2025).

[3] J.-G. Dumas, C. Pernet, A. Sedoglavic, *A more accurate rational non-commutative algorithm for multiplying* 4×4 *matrices using* 48 *multiplications*, arXiv:2603.18699 (v1 19 March 2026, v3 29 July 2026).

[4] J.-G. Dumas, C. Pernet, A. Sedoglavic, *Towards automated generation of fast and accurate algorithms for recursive matrix multiplication*, J. Symbolic Comput. 134 (2026) 102524.

[5] J.-G. Dumas, C. Pernet, A. Sedoglavic, J. Stapleton, P. Tichavský, *Fast matrix multiplication via recursive* $\langle 4 \times 4 \times 4 : 48 \rangle$ *algorithms into practice*, arXiv:2609.12027 (v1 10 September 2026, v2 2 October 2026).

[6] J.-G. Dumas, B. Grenet, C. Pernet, A. Sedoglavic, *PLinOpt, a collection of C++ routines handling linear & bilinear programs*, https://github.com/jgdumas/plinopt; data files `4x4x4_48_accurate_262` (commit `b441528`, 13 October 2026), `3x3x6_40_DPS-accurate`, `3x6x3_40_DPS-accurate`, `6x3x3_40_DPS-accurate`; tools `optimizer`, `transpozer`, `SLPchecker`.

[7] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002 (Chapter 23, fast matrix multiplication).

[8] J. Huang, T. M. Smith, G. M. Henry, R. A. van de Geijn, *Strassen's algorithm reloaded*, Proc. SC '16 (International Conference for High Performance Computing, Networking, Storage and Analysis), 2016.

[9] J. Huang, C. D. Yu, R. A. van de Geijn, *Strassen's algorithm reloaded on GPUs*, ACM Trans. Math. Softw. 46 (2020), 1:1–1:22.

[10] M. Kauers, J. Wood, *Consequences of the Moosbauer–Poole algorithms*, preprint (2025).

[11] A. Lavin, S. Gray, *Fast algorithms for convolutional neural networks*, Proc. CVPR 2016, 4013–4021.

[12] J. Moosbauer, M. Poole, *Flip graphs with symmetry and new matrix multiplication schemes*, preprint (2025).

[13] E. Mårtensson, P. Stankovski Wagner, *The number of the beast: reducing additions in fast matrix multiplication algorithms for dimensions up to 666*, Proc. ACDA 2025; the library `fmm_add_reduction`, https://github.com/werekorren/fmm_add_reduction (commit `951cd93`, 8 May 2025), whose `algorithms/` directory distributes the Kauers–Wood, Moosbauer–Poole–Kemper and AlphaEvolve schemes used in the screen.

[14] A. Novikov et al., *AlphaEvolve: a coding agent for scientific and algorithmic discovery*, arXiv:2506.13131 (2025).

[15] K. Ozaki, T. Ogita, S. Oishi, S. M. Rump, *Error-free transformations of matrix multiplication by using fast routines of matrix multiplication and its applications*, Numer. Algorithms 59 (2012), 95–118.

[16] A. I. Perminov, *FastMatrixMultiplication*: a catalogue of fast matrix multiplication schemes, https://github.com/dronperminov/FastMatrixMultiplication, commit `3ee3284` (28 October 2026); file `schemes/results/addition_reduced_ZT/3x3x6_m42_cr139_cn287_ZT_reduced.json`.

[17] A. I. Perminov, *Parallel heuristic exploration for additive complexity reduction in fast matrix multiplication*, arXiv:2512.13365 (2025).

[18] A. V. Smirnov, *The bilinear complexity and practical algorithms for matrix multiplication*, Comput. Math. Math. Phys. 53 (2013) 1781–1795.

[19] G. Sidebottom, *201 operations for rank-48* 4×4 *matrix multiplication over rings containing* $1/2$, preprint (2026); PLinOpt data file `4x4x4_48_201`.

[20] G. Sidebottom, *The 262-operation* $\langle 4 \times 4 \times 4 : 48\rangle$ *algorithm is also the more accurate one: error-bound exponent* $9/4$ *and about a quarter less error*, draft of 27 October 2026, `doc/matmul_4x4_rank48_accurate262.pdf` in [21].

[21] G. Sidebottom, *logic*, source repository, https://github.com/gsidebottom/logic (2026).

n, precision	plan	time (s)	over GEMM	relerr	err64
6912 f32	`G` (Accelerate)	0.195	1.00	–	$3.85 \cdot 10^{-6}$
	`D48/G`	0.154	1.27	$4.2 \cdot 10^{-6}$	$2.81 \cdot 10^{-6}$
	`R336/G`	0.164	1.19	$3.9 \cdot 10^{-6}$	$2.27 \cdot 10^{-6}$
	`R363/G`	0.158	1.23	$4.2 \cdot 10^{-6}$	$1.76 \cdot 10^{-6}$
	`D48+4/D48/G6`	0.149	1.31	$4.3 \cdot 10^{-6}$	$2.94 \cdot 10^{-6}$
	`R363+4/D48/G6` (auto)	**0.141**	**1.38**	$4.0 \cdot 10^{-6}$	$2.26 \cdot 10^{-6}$
13824 f32	`G` (Accelerate)	1.565	1.00	–	$6.40 \cdot 10^{-6}$
	`D48/G`	1.255	1.25	$7.1 \cdot 10^{-6}$	$4.16 \cdot 10^{-6}$
	`R336/G`	1.244	1.26	$6.7 \cdot 10^{-6}$	$3.80 \cdot 10^{-6}$
	`R363/G`	1.242	1.26	$6.7 \cdot 10^{-6}$	$2.73 \cdot 10^{-6}$
	`R336/R363/G`	1.200	1.30	$6.2 \cdot 10^{-6}$	$3.34 \cdot 10^{-6}$
	`D48+4/D48/G6`	0.949	1.65	$7.0 \cdot 10^{-6}$	$4.35 \cdot 10^{-6}$
	`R363+3/D48/G6`	0.906	1.73	$6.4 \cdot 10^{-6}$	$3.49 \cdot 10^{-6}$
	`R363+4/R363/G6` (auto)	**0.907**	**1.73**	$6.6 \cdot 10^{-6}$	$2.46 \cdot 10^{-6}$
6912 f64	`G` (Accelerate)	0.773	1.00	–	$6.69 \cdot 10^{-15}$
	`D48/G`	0.612	1.26	$7.8 \cdot 10^{-15}$	$4.89 \cdot 10^{-15}$
	`R336/G`	0.597	1.29	$6.8 \cdot 10^{-15}$	$4.53 \cdot 10^{-15}$
	`R363/G`	0.647	1.19	$6.8 \cdot 10^{-15}$	$2.86 \cdot 10^{-15}$
	`R336/R363/G`	0.594	1.30	$8.3 \cdot 10^{-15}$	$3.66 \cdot 10^{-15}$
	`D48+4/D48/G6`	0.478	1.62	$7.9 \cdot 10^{-15}$	$5.60 \cdot 10^{-15}$
	`R363+3/R633/G6` (auto)	**0.451**	**1.71**	$6.7 \cdot 10^{-15}$	$3.75 \cdot 10^{-15}$
13824 f64	`G` (Accelerate)	6.223	1.00	–	$1.23 \cdot 10^{-14}$
	`D48/G`	4.970	1.25	$1.40 \cdot 10^{-14}$	$7.80 \cdot 10^{-15}$
	`R336/G`	4.946	1.26	$1.18 \cdot 10^{-14}$	$7.21 \cdot 10^{-15}$
	`R363/G`	4.919	1.27	$1.30 \cdot 10^{-14}$	$4.97 \cdot 10^{-15}$
	`R336/R363/G`	3.703	1.68	$1.45 \cdot 10^{-14}$	$6.72 \cdot 10^{-15}$
	`D48+4/D48/G6`	3.703	1.68	$1.38 \cdot 10^{-14}$	$8.28 \cdot 10^{-15}$
	`R363+3/D48/G6`	3.467	1.80	$1.25 \cdot 10^{-14}$	$5.81 \cdot 10^{-15}$
	`R363+3/R336/G3` (auto)	3.296	1.89	$1.33 \cdot 10^{-14}$	$6.40 \cdot 10^{-15}$
	`SW/D48+2/D48/G6`	3.407	1.83	$1.85 \cdot 10^{-14}$	$1.72 \cdot 10^{-14}$
	`SW/R363+3/R633/G6` (no budget)	**3.251**	**1.91**	$1.51 \cdot 10^{-14}$	$1.25 \cdot 10^{-14}$

Table 5: Square problems on the M4 Pro (`results/r336_campaign2_2026-10-28.txt`; the `R363/G` rows and the 13824 f64 `R363+3/D48/G6` row from `t42_campaign` and `f64exact` of the same day): best of three to five interleaved rounds; "over GEMM" is Accelerate's time divided by the plan's; relerr is the max-norm difference from Accelerate's product; err64 the max-norm error against the exact product – the f64 product in f32, the compensated product in f64 (`results/f64exact_2026-10-28.txt`) – with the GEMM's own error in the first row of each block. "auto" is the planner's choice under its default budget (the 13824 f64 `R363+3/R336/G3` row is from the budget campaign, Table 6). Only err64 compares plans: the GEMM's own error is as large as a level's, so relerr mixes the two (`R336/G` and `R363/G` at 6912 f64 have the same relerr and exact errors 4.53 and $2.86 \cdot 10^{-15}$).

$m \times k \times n$	calibration	prec.	plan	predicted / measured (s)	over GEMM
$3456 \times 3456 \times 1728$	for the shape	f32	`R363/G6`	0.0117 / 0.0115	1.05
		f64	`R363/G6`	0.0366 / 0.0367	1.30
$1000 \times 3000 \times 2000$	estimated	f32	`G`	–	1.00
		f64	`SW/G4`	0.0133 / 0.0144	1.04
$2048 \times 4096 \times 1024$	estimated	f32	`G`	–	1.00
		f64	`D48/G6`	0.0177 / 0.0180	1.10
$6912 \times 6912 \times 3456$	estimated	f32	`D48/G3`	0.0812 / 0.0777	1.30
		f64	`D48+4/R363/G6`	0.2578 / 0.2565	1.52

Table 7: Rectangular problems through the library's entry points (`results/rect_api_2026-10-28.txt`): the planner's plan, its predicted and measured time, and the speedup over Accelerate's GEMM on the same shape. "Estimated" shapes are priced from the square calibration by (7). The plans are the default budget's; without one the f32 $6912 \times 6912 \times 3456$ plan is `SW+4/D48/G6`, 0.0790 / 0.0796 s, 1.26×.

	GPU, MPSGraph		Accelerate	GPU and Acc.	best mixed plan		
n (f32)	time (s)	TFLOP/s	time (s)	error	plan	time (s)	error
3456	0.013	6.3	0.025	$3.04 \cdot 10^{-6}$	`R363/G6`	0.021	$1.28 \cdot 10^{-6}$
6912	0.099	6.6	0.195	$3.85 \cdot 10^{-6}$	`R363+4/D48/G6`	0.141	$2.26 \cdot 10^{-6}$
13824	0.946	5.6	1.565	$6.40 \cdot 10^{-6}$	`R363+4/R363/G6`	0.907	$2.46 \cdot 10^{-6}$

Table 8: The GPU as a GEMM (`results/gpu_gemm_2026-10-28.txt`): best of five rounds; the GPU's f32 error equals Accelerate's, hence one column. With f16 inputs and an f32 result the GPU takes 0.011, 0.090 and 0.790 s with errors 2.5–$2.8 \cdot 10^{-4}$. Accelerate's and the plans' numbers are the campaigns' (Table 5, and the 3456 exact errors of `results/f64exact_2026-10-28.txt`).

n, prec.	plan	R (s)	T (s)	T/R	R error	T error	T/R
13824 f32	`X336/G`	1.228	1.286	1.05	3.80e-6	1.28e-5	3.4
	`X363/G`	1.242	1.304	1.05	2.73e-6	6.25e-6	2.3
	`X633/G`	1.276	1.339	1.05	3.95e-6	1.01e-5	2.6
	`X336/X363/X633/G`	2.538	2.874	1.13	4.95e-6	3.53e-5	7.1
	`X363+4/D48/G6`	0.909	0.954	1.05	3.49e-6	6.23e-6	1.8
	`X363+4/X363/G6`	0.906	0.994	1.10	2.46e-6	9.04e-6	3.7
13824 f64	`X336/G`	4.858	5.089	1.05	1.18e-14	2.62e-14	2.2
	`X363/G`	4.919	5.132	1.04	1.30e-14	1.34e-14	1.0
	`X336/X363/X633/G`	7.546	8.763	1.16	1.24e-14	8.08e-14	6.5
	`SW/X363+3/X633/G6`	3.236	3.573	1.10	1.51e-14	5.61e-14	3.7
	`X363+3/X633/G6`	5.624	6.188	1.10	1.26e-14	2.63e-14	2.1
6912 f32	`X336/G`	0.165	0.173	1.05	2.27e-6	8.00e-6	3.5
	`X363/G`	0.158	0.160	1.01	1.76e-6	3.87e-6	2.2
	`X363+4/D48/G6`	0.142	0.147	1.04	2.26e-6	4.07e-6	1.8
	`X363+3/X633/G6`	0.160	0.173	1.08	2.20e-6	8.72e-6	4.0
6912 f64	`X336/G`	0.598	0.626	1.05	6.78e-15	1.47e-14	2.2
	`X363+4/D48/G6`	0.453	0.478	1.06	7.03e-15	8.81e-15	1.3
	`X363+3/X633/G6`	0.451	0.497	1.10	6.74e-15	1.63e-14	2.4

Table 9: The ternary $\langle 3 \times 3 \times 6 : 42 \rangle$ levels (T) against the accurate $\langle 3 \times 3 \times 6 : 40 \rangle$ levels (R) on the same plans (`results/t42_campaign_2026-10-28.txt`; `X` stands for `R` or `T`): best of five (f32) or three (f64) interleaved rounds, seconds; the f32 error is against the f64 product, the f64 error against Accelerate's product, which in f64 includes Accelerate's own error, as large as an R level's (Section 6), so the f32 ratios carry the comparison.